Open Source Software Audit Report Visual Studio Code

Name: Samay Rajput
Reg. no.: 24BCG10018
Course: Open Source Software
Environment: Arch-based Linux (Omarchy)

1. Introduction

Software that anybody may use, alter, and share without limitations is known as open source software. Because it supports operating systems, development tools, and web technologies, it is crucial for modern computers. The origin, license, ethical issues, Linux footprint, and ecosystem of Visual Studio Code are the main subjects of this report's assessment.

2. Part A — Origin and Philosophy

2.1 Origin Story

In 2015, Microsoft launched Visual Studio Code, a lightweight, cross-platform code editor. Before its launch, developers faced a choice between rudimentary editors lacking features and sophisticated integrated development environments (IDEs). This was remedied by VS Code, which acquired popular use by merging performance with extensibility via plugins.

Before the launch of Visual Studio Code, developers often had to decide between heavy integrated development environments (IDEs) like Eclipse and IntelliJ, or lightweight editors like

Notepad++ that lacked complex capabilities. This created a gap in the developer experience, where tools were either too complex or too restrictive.

Microsoft introduced Visual Studio Code in 2015 to overcome this issue. It was supposed to be lightweight, rapid, and immensely extendable. Unlike other IDEs, VS Code focuses on modularity, allowing developers to create modules based on their needs. This made it appropriate for many programming languages and approaches.

Another crucial feature of VS Code was its cross-platform portability. It operates wonderfully on Windows, Linux, and macOS, which made it universally accessible. By open-sourcing the core (Code-OSS), Microsoft supported community contributions, resulting to quick development and innovation.

The decision to make it open source shows a transition in Microsoft's mentality, from a typically proprietary firm to one that actively engages in the open-source environment.

2.2 License Analysis

VS Code (Code-OSS) is licensed under the MIT License, which permits free use, modification, and dissemination. Unlike GPL, MIT does not require modified versions to stay open source. This allows flexibility but less protection for community contributions.

The MIT License, which is a liberal open-source license, lets you utilize Visual Studio Code (Code-OSS).

This implies that users can use, update, distribute, and even sell the application without many limitations.

The MIT License grants you the four core freedoms of free software: the right to execute the program for any reason, learn how it works, change it, and share copies. The MIT License, on the other hand, does not oblige modified versions to stay open source as the GNU General Public License (GPL) does.

This is a significant contrast between permissive and copyleft licensing. The GPL preserves community contributions and keeping software open, but the MIT license allows developers and enterprises more flexibility. This enables organizations develop their own versions of VS Code based on the open-source core.

This makes me think about the contrast between freedom and control. MIT wants a lot more people to utilize its technology, but it might also help firms generate money without contributing back to the community.

2.3 Ethics of Open Source

Open source makes things more open and encourages people to work together. But open-source work can also help enterprises make money. Microsoft contributes to open source in the case of VS Code, but it still controls how it is publicly delivered.

Open source software is more than simply a technique to produce software; it is also a way of thinking about sharing knowledge and working together. It helps developers from all around the world work together, learn from each other's work, and build on it.

But it's up for dispute whether all software should be open source. Open source, on the other hand, supports security, openness, and fresh ideas. Companies, on the other hand, put a lot of money into development and may need proprietary control to keep their business models continuing.

Another ethical dilemma is if it's fair for enterprises to make money off of contributions to open source software.

A lot of major corporations produce things employing open-source software without contributing anything back. This is permitted under permissive licenses, but it makes many question about justice and long-term survival. "Standing on the shoulders of giants" is a tagline that goes nicely with open source.

Every current tool is built on top of work that has already been done. Open source makes it easy for developers to swiftly come up with new ideas by allowing them build on and improve existing ones instead of beginning from zero.

3. Part B — Linux Footprint

The investigation was done on an Arch-based Linux system dubbed Omarchy. To use VS Code, you need to install it and then type "code" in the command line. The configuration files are kept at ~/.config/Code. The system package manager can be used to update the software, which runs under the current user.

We did the analysis of Visual Studio Code on an Arch-based Linux system dubbed Omarchy. Arch used pacman as its package manager instead of dpkg, which meant that commands had to be used in a somewhat different way.

The VS Code executable is generally stored under /usr/bin/code, which makes it available to the whole system. The user directory has configuration files in ~/.config/Code. This enables individuals alter their personal settings without impacting the general system.

The application runs as the user who is now signed in, which is crucial for security since it forbids anybody from logging into the system without authority. VS Code does not operate as a background daemon as system services do, such as web servers.

You may handle updates using the package manager or through alternative repositories like AUR (Arch User Repository). This demonstrates how open-source software distribution is decentralized, with updates being pushed by the community and maintainers.

4. Part C — FOSS Ecosystem

VS Code leverages technologies like Electron, Node.js, and Chromium. It has a variety of extensions that make it more helpful and help developers get more done.

Visual Studio Code is part of a wide open-source ecosystem. It leverages Electron to make web technologies work on desktop programs, Node.js to run the code, and Chromium to render the pages.

The extension ecosystem is one of the nicest things about VS Code. Extensions for languages, themes, debugging tools, and integrations can be created by developers. This modular design allows VS Code a lot of flexibility and adaptation.

GitHub is where the project is actively maintained.

Developers may work on it, report mistakes, and suggest approaches to make it better. This community-driven approach makes sure that things keep developing and getting better.

VS Code also interacts with technologies like Git, Docker, and cloud platforms, thus it's an essential element of contemporary development workflows.

5. Part D — Comparison

You can compare VS Code to Sublime Text. Sublime Text is a premium program that is controlled by a firm, while VS Code is free and open source with a lot of add-ons. VS Code allows you tweak things more and offers better community support.

You may compare Visual Studio Code to Sublime Text, which is a code editor that you have to buy. Sublime Text is recognized for being quick and easy to use, however it doesn't provide as many extensions as VS Code does.

You don't have to pay for Sublime Text to keep using it, but you do have to pay for VS Code. VS Code is more configurable as it enables you develop extensions and adjust settings.

Open-source software like VS Code allows developers look at the source code, which is not available with private software. This is better for security. But proprietary tools might offer more stable and controlled support.

Most developers prefer VS Code because it has a superior blend of functionality, affordability, and community support.

6. Shell Scripts

Script 1: System Identity Report

Code:

```
#!/bin/bash

# Script 1: System Identity Report
# Author: Samay Rajput| Course: Open Source Software

# --- Variables ---
STUDENT_NAME="Samay Rajput"
SOFTWARE_CHOICE="Visual Studio Code"

# --- System Info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DATE=$(date)
HOME_DIR=$HOME
DISTRO=$(lsb_release -d 2>/dev/null | cut -f2)

# --- Display ---
echo "========================================"
echo " Open Source Audit — $STUDENT_NAME"
echo "========================================"
echo "Software Chosen : $SOFTWARE_CHOICE"
echo "Kernel Version : $KERNEL"
echo "User : $USER_NAME"
echo "Home Directory : $HOME_DIR"
echo "Uptime : $UPTIME"
echo "Date & Time : $DATE"
echo "Distro : $DISTRO"
echo "License : Linux OS is generally under GPL license"
echo "========================================"
```

Output:

```
$ ./script1.sh
./script1.sh: line 12: uptime: command not found
========================================
 Open Source Audit — Samay Rajput
========================================
Software Chosen : Visual Studio Code
Kernel Version  : 3.6.6-1cdd4371.x86_64
User            : rajpu
Home Directory  : /c/Users/rajpu
Uptime          :
Date & Time     : Tue, Mar 31, 2026  9:43:42 AM
Distro          :
License         : Linux OS is generally under GPL license
========================================
```

Explanation:

This script creates a system identity report by providing crucial facts about the Linux system. It fetches information such the version of the kernel, the current user, the home directory, the system's uptime, the date and time, and the Linux distribution. There is also a line in the script detailing the system's open-source license.

It explains how to utilize variables, command substitution ($()), and echo to format output. This script explains how to utilize shell scripting to get to and present system-level information.

Script 2: FOSS Package Inspector

Code:

```
#!/bin/bash

# Script 2: FOSS Package Inspector (Clean Version)

PACKAGE="code"
```

```
echo "Checking package: $PACKAGE"
echo "-------------------------"

# Check if VS Code is installed
if command -v code &> /dev/null; then
echo "VS Code is installed."

# Show version
code --version | head -n 1

# Show path
echo "Location: $(which code)"
else
echo "VS Code is NOT installed."
fi

# Case statement
case $PACKAGE in
code) echo "VS Code: a lightweight, extensible open-source editor" ;;
git) echo "Git: version control system built for collaboration" ;;
nginx) echo "Nginx: high-performance web server" ;;
python) echo "Python: powerful open-source programming language" ;;
*) echo "Unknown package" ;;
esac
```

Output:

```
$ ./script2.sh
Checking package: code
-------------------------
VS Code is installed.
1.113.0
Location: /c/Users/Asus/AppData/Local/Programs/Microsoft VS Code/bin/code
VS Code: a lightweight, extensible open-source editor
```

Explanation:

This script checks to see if Visual Studio Code is on the PC. It checks to discover if the code command is available by performing system commands. If it is, it shows the version and installation path. It will let you know if the program isn't installed.

Also, a case statement is used to explain how conditional logic works by printing a short explanation of the software. The script explains how package inspection might be different for different Linux distributions. For example, it was updated for an Arch-based system to use the right instructions instead of dpkg.This script checks to see if Visual Studio Code is on the PC. It checks to determine if the code command is accessible by running system commands. If it is, it shows the version and installation path. It will let you know if the software isn't installed.

Also, a case statement is used to explain how conditional logic works by printing a short explanation of the software. The script explains how package inspection might be different for different Linux distributions. For example, it was updated for an Arch-based system to use the right instructions instead of dpkg.

Script 3: Disk and Permission Auditor

Code:

```
#!/bin/bash

# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------"
```

```
# Loop through directories
for DIR in "${DIRS[@]}"; do
if [ -d "$DIR" ]; then
PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)

echo "$DIR => Permissions: $PERMS | Size: $SIZE"
else
echo "$DIR does not exist"
fi
done

echo "----------------------"

# VS Code config directory check
CONFIG_DIR="$HOME/.config/Code"

if [ -d "$CONFIG_DIR" ]; then
PERMS=$(ls -ld $CONFIG_DIR | awk '{print $1, $3, $4}')
echo "VS Code Config Found:"
echo "$CONFIG_DIR => $PERMS"
else
echo "VS Code config directory not found"
fi
```

Output:

```
$ ./script6.sh
$ ./script3.sh
Directory Audit Report

./script3.sh: line 12: echo/etc => Permissions: drwxr-xr-x Samay Rajput 197121 | Size: 1.6M
/var/log does not exist
/home does not exist
/usr/bin => Permissions: drwxr-xr-x Samay Rajput 197121 | Size: 87M
/tmp => Permissions: drwxr-xr-x Samay Rajput 197121 | Size: 288M: No such file or directory

VS Code config directory found:
/home/frosfres/.config/Code => drwx ---

frosfres frosfres
```

Explanation:

The permissions and disk space of important system directories, such as /etc, /var/log, /home, /usr/bin, and /tmp, are checked by this script. It uses a for loop to go through each of these files one at a time and collects data on their ownership, rights, and disk space usage.
To gather the data it requires, the script makes use of text processing programs like awk and commands like ls -ld and du -sh. Additionally, it determines if the user's home folder contains the Visual Studio Code configuration directory, demonstrating how to manage conditional checks and directory validation.

Script 4: Log File Analyzer

Code:

```
#!/bin/bash

# Script 4: Log File Analyzer

LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0
```

```
# Check file exists
if [ ! -f "$LOGFILE" ]; then
echo "Error: File $LOGFILE not found."
exit 1
fi

# If file empty → retry (simple loop)
while [ ! -s "$LOGFILE" ]; do
echo "File is empty. Waiting..."
sleep 2
done

# Read file line by line
while IFS= read -r LINE; do
if echo "$LINE" | grep -iq "$KEYWORD"; then
COUNT=$((COUNT + 1))
fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"

echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

Output:

```
$ ./script4.sh
Keyword 'error' found 6 times in /var/log/pacman. log
Last 5 matching lines:
[2026-01-21T16:50:55+0000] [ALPM] installed perl-error (0.17030-3)
[2026-01-21T22:21:57+0530] [ALPM] installed xcb-util-errors (1.0.1-2)
[2026-01-30T00:07:53+0530] [ALPM] installed lib32-libgpg-error (1.58-1)
[2026-02-24T12:30:40+0530] [ALPM] upgraded Libgpg-error (1.58-1 > 1.59-1)
[2026-02-24T12:30:43+0530] [ALPM] upgraded Lib32-Libgpg-error (1.58-1 > 1.59-1)
```

Explanation:

The script also shows the last five matching lines with grep and tail. This offers a brief summary of the log entries that are significant. The script was tested using /var/log/pacman.log instead of /var/log/syslog since the system used in this project is Arch-based (Omarchy).

Script 5: The Open Source Manifesto Generator

Code:

```
#!/bin/bash

# Script 5: Open Source Manifesto Generator

echo "Answer three questions to generate your manifesto."
echo ""

# Taking input
read -p "1. One open-source tool you use daily: " TOOL
read -p "2. One word for freedom: " FREEDOM
read -p "3. One thing you would build: " BUILD

DATE=$(date '+%d %B %Y')
USER=$(whoami)
OUTPUT="manifesto_$USER.txt"

# Creating manifesto
echo "Open Source Manifesto" > $OUTPUT
echo "Date: $DATE" >> $OUTPUT
echo "" >> $OUTPUT
echo "I use $TOOL every day, which represents the power of open
collaboration." >> $OUTPUT
echo "For me, freedom means $FREEDOM in the digital world." >>
$OUTPUT
echo "In the future, I aim to build $BUILD and share it openly." >>
$OUTPUT
```

```
echo "I believe open source drives innovation and collective growth."
>> $OUTPUT

echo ""
echo "Manifesto saved to $OUTPUT"
echo "-------------------------"
cat $OUTPUT
```

Output:

```
$ ./script5.sh
Answer three questions to generate your manifesto.

1. One open-source tool you use daily: VS code
2. One word for freedom: Modi
3. One thing you would build: developer tools

Manifesto saved to manifesto_Eesh.txt
-------------------------
Open Source Manifesto
Date: 28 March 2026

I use VS code every day, which represents the power of open collaboration.
For me, freedom means Modi in the digital world.
In the future, I aim to build developer tools and share it openly.
I believe open source drives innovation and collective growth.
```

Explanation:

This program develops a customized open-source manifesto by conversing to the user via the terminal. It asks the user three questions and records their responses in variables.
The script then puts these inputs together into a structured paragraph and saves it to a text file by leveraging output redirection (> and >>). It also shows the stuff that was created on the terminal. This script explains how to handle user input, alter strings, and write to files with shell scripting.

Open Source vs Proprietary Comparison

Visual Studio Code vs Sublime Text

Feature	Visual Studio Code (Open Source)	Sublime Text (Proprietary)
Cost	Free	Paid (trial available)
Source Code	Open (Code-OSS)	Closed
Security	Can be audited by anyone	Cannot be inspected
Customization	Highly customizable via extensions	Limited customization
Extensions	Huge marketplace	Smaller ecosystem
Performance	Slightly heavier (Electron-based)	Very fast and lightweight
Updates	Frequent community-driven updates	Controlled by company
Support	Community + documentation	Official support available
Control	Community + Microsoft	Fully company controlled

Visual Studio Code and Sublime Text are two different sorts of software development tools. VS Code is an open-source editor that

may be changed and enhanced by the community. Sublime Text, on the other hand, is a proprietary software that is produced and supported by one firm.

The fact that VS Code is open source is one of its best advantages. Developers can look at the source code, change it, and help make it better. This transparency promotes trust and speeds up innovation. Sublime Text, on the other hand, doesn't let you examine its source code, which makes it less open and less customisable at a deeper level.

VS Code is free, which implies that anyone can use it, from students to independent developers. Sublime Text has a free trial, but you need to pay for a license to keep using it. This may be a problem for some people.

But Sublime Text does offer some good points. It is noted for being fast and efficient because it doesn't need hefty frameworks like Electron. This makes it lighter and more responsive than VS Code, which can require more system resources.

VS Code has a significantly wider extension environment, which lets developers add support for practically any language or tool. VS Code is suitable for modern development workflows since it is so flexible. Sublime Text also offers plugins, although there aren't as many of them and they aren't as active.

From a control point of view, both the open-source community and Microsoft have an effect on VS Code. On the other hand, Sublime Text is totally controlled by its developers. This highlights the contrast between software development approaches that are driven by the community and those that are led by companies.

Most developers choose Visual Studio Code because it offers a better mix between flexibility, pricing, and community support. But

Sublime Text is still a fantastic alternative for folks who want things to be fast and easy.

Visual Studio Code and Sublime Text are two different sorts of software development tools. VS Code is an open-source editor that may be changed and enhanced by the community. Sublime Text, on the other hand, is a proprietary software that is produced and supported by one firm.

The fact that VS Code is open source is one of its best advantages. Developers can look at the source code, change it, and help make it better. This transparency promotes trust and speeds up innovation. Sublime Text, on the other hand, doesn't let you examine its source code, which makes it less open and less customisable at a deeper level.

VS Code is free, which implies that anyone can use it, from students to independent developers. Sublime Text has a free trial, but you need to pay for a license to keep using it. This may be a problem for some people.

But Sublime Text does offer some good points. It is noted for being fast and efficient because it doesn't need hefty frameworks like Electron. This makes it lighter and more responsive than VS Code, which can require more system resources.

VS Code has a significantly wider extension environment, which lets developers add support for practically any language or tool. VS Code is suitable for modern development workflows since it is so flexible. Sublime Text also offers plugins, although there aren't as many of them and they aren't as active.

When it comes to control, both the open-source community and Microsoft have an impact on VS Code. On the other hand, Sublime Text is totally controlled by its developers. This highlights the

distinction between software development models that are driven by the community and those that are driven by the company.

Overall, Visual Studio Code provides a better blend of versatility, pricing, and community support, making it a favored choice for most developers. However, Sublime Text remains a solid alternative for consumers who favor speed and simplicity.

Conclusion:

This research provides vital insights on the importance of open-source software in modern computing. Visual Studio Code highlights how open-source ideas may be paired with corporate support to produce strong and widely adopted tools.

Through this audit, it became evident that open source is not just about free software, but about collaboration, transparency, and shared progress. Understanding these ideas is vital for any software developer.